Multi-Scanner Correlation Strategies

DevOps.ai

January 2025

Contents

1 Multi-Scanner Correlation Strategies

1.1 How to Effectively Correlate Findings from Multiple Security Scanners

Version 1.0 | **January 2025** | **DevOps.ai**

1.2 Executive Summary

Modern security programs use multiple scanners (SAST, DAST, SCA, CNAPP) to achieve comprehensive coverage. However, overlapping findings create alert fatigue, duplicate remediation efforts, and obscure true risk. Organizations report 40-60% duplicate findings across scanners, overwhelming security teams with noise.

This whitepaper provides practical strategies for correlating findings from multiple security scanners to reduce false positives, improve remediation prioritization, and achieve unified risk visibility using AlDeci.

Key Benefits: - Reduce duplicate findings by 40-60% - Improve remediation prioritization with unified risk scores - Achieve single pane of glass for security findings - Reduce MTTR by 46% through context-aware correlation

---

1.3 Table of Contents

1.4 1. The Multi-Scanner Challenge

1.4.1 1.1 Why Multiple Scanners?

Organizations use multiple security scanners for comprehensive coverage:

SAST (Static Application Security Testing) - Purpose: Find vulnerabilities in source code - Examples: SonarQube, Semgrep, Checkmarx, Fortify - Coverage: Code-level vulnerabilities (SQL injection, XSS, etc.)

DAST (Dynamic Application Security Testing) - Purpose: Find vulnerabilities in running applications - Examples: OWASP ZAP, Burp Suite, Acunetix - Coverage: Runtime vulnerabilities, configuration issues

SCA (Software Composition Analysis) - Purpose: Find vulnerabilities in dependencies - Examples: Snyk, WhiteSource, Black Duck, Dependabot - Coverage: Open-source vulnerabilities, license compliance

CNAPP (Cloud-Native Application Protection Platform) - Purpose: Find vulnerabilities in cloud infrastructure - Examples: Wiz, Orca, Prisma Cloud, Aqua Security - Coverage: Container vulnerabilities, misconfigurations, runtime threats

IAST (Interactive Application Security Testing) - Purpose: Find vulnerabilities during testing - Examples: Contrast Security, Veracode IAST - Coverage: Runtime vulnerabilities with code-level context

1.4.2 1.2 The Overlap Problem

Scenario: Organization uses Snyk (SCA), SonarQube (SAST), and Wiz (CNAPP)

Finding 1 (Snyk):

```
{
  "tool": "Snyk",
  "type": "SCA",
  "vulnerability": "CVE-2021-44228",
  "component": "log4j-core",
  "version": "2.14.1",
  "severity": "CRITICAL",
  "cvss": 10.0
}
```

Finding 2 (SonarQube):

```
{
  "tool": "SonarQube",
  "type": "SAST",
  "vulnerability": "Vulnerable Dependency",
  "component": "log4j-core:2.14.1",
  "severity": "CRITICAL",
  "rule": "java:S4347"
}
```

Finding 3 (Wiz):

```
{
  "tool": "Wiz",
  "type": "CNAPP",
  "vulnerability": "CVE-2021-44228",
  "package": "org.apache.logging.log4j:log4j-core:2.14.1",
  "severity": "CRITICAL",
  "cvss": 10.0
}
```

Problem: Same vulnerability reported 3 times by 3 different tools!

Impact: - 3 separate tickets created in Jira - 3 separate remediation efforts - 3 separate evidence bundles - Security team overwhelmed with duplicates

1.4.3 1.3 The False Positive Problem

Scenario: SAST tool reports SQL injection vulnerability

SAST Finding:

```
{
  "tool": "SonarQube",
  "type": "SAST",
  "vulnerability": "SQL Injection",
  "file": "src/db/query.java",
  "line": 42,
  "severity": "HIGH",
```

```
  "code": "String query = \"SELECT * FROM users WHERE id = \" + userId;"
}
```

Context (missed by SAST): - `userId` is validated by input sanitization function - Database uses parameterized queries in production - Code path is only reachable by authenticated admins

Result: False positive (or very low risk)

Problem: SAST tools lack runtime context, leading to high false-positive rates (30-50%).

1.5 2. Correlation Fundamentals

1.5.1 2.1 Correlation Dimensions

AlDeci correlates findings across 6 dimensions:

1. Vulnerability Identity - CVE ID (e.g., CVE-2021-44228) - CWE ID (e.g., CWE-89 for SQL injection) - GHSA ID (GitHub Security Advisory)

2. Component Identity - Package name (e.g., log4j-core) - Package version (e.g., 2.14.1) - Package URL (purl) (e.g., pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1)

3. Location - File path (e.g., src/db/query.java) - Line number (e.g., line 42) - Function/method name (e.g., getUserById)

4. Severity - CVSS score (0.0-10.0) - Scanner-specific severity (Critical, High, Medium, Low) - Risk level (FixOpsRisk 0-100)

5. Remediation - Fix version (e.g., upgrade to 2.23.1) - Remediation guidance (e.g., apply patch, change configuration) - Workaround (e.g., disable vulnerable feature)

6. Metadata - Scanner tool (e.g., Snyk, SonarQube, Wiz) - Scan type (SAST, DAST, SCA, CNAPP) - Scan timestamp - Project/component name

1.5.2 2.2 Correlation Strategies

Strategy 1: Exact Match - Match on CVE ID + component + version - High confidence (99%+ accuracy) - Misses findings without CVE IDs

Strategy 2: Fuzzy Match - Match on component name + version (ignore CVE) - Medium confidence (80-90% accuracy) - Catches findings with different CVE IDs for same vulnerability

Strategy 3: Semantic Match - Match on vulnerability description using NLP - Low confidence (60-70% accuracy) - Catches findings with no CVE or component info

Strategy 4: Location Match - Match on file path + line number - High confidence for SAST findings - Not applicable for SCA/CNAPP findings

AIDeci Approach: Hybrid strategy using all 4 methods with confidence scoring.

1.5.3 2.3 Confidence Scoring

AIDeci assigns confidence scores to correlations:

Confidence Formula:

```
confidence = (
    0.40 * cve_match_score +
    0.30 * component_match_score +
    0.15 * location_match_score +
    0.10 * severity_match_score +
    0.05 * metadata_match_score
)
```

Confidence Levels: - **High (≥ 0.9)**: Exact CVE + component + version match - **Medium (0.7-0.9)**: Fuzzy component match or semantic match - **Low (< 0.7)**: Weak semantic match or location-only match

Action Based on Confidence: - High confidence: Automatic deduplication - Medium confidence: Flag for human review - Low confidence: Keep separate findings

---

1.6 3. Deduplication Algorithms

1.6.1 3.1 Algorithm 1: CVE-Based Deduplication

Input: Multiple findings with same CVE ID

Algorithm:

```
def deduplicate_by_cve(findings):
    """Deduplicate findings by CVE ID"""
    cve_groups = {}

    for finding in findings:
        cve_id = finding.get('cve_id')
        if not cve_id:
            continue

        if cve_id not in cve_groups:
            cve_groups[cve_id] = []

        cve_groups[cve_id].append(finding)

    deduplicated = []
    for cve_id, group in cve_groups.items():
        # Merge findings with same CVE
        merged = merge_findings(group)
```

```
        deduplicated.append(merged)

    return deduplicated

def merge_findings(findings):
    """Merge multiple findings into one"""
    merged = {
        'cve_id': findings[0]['cve_id'],
        'component': findings[0]['component'],
        'version': findings[0]['version'],
        'severity': max([f['severity'] for f in findings]),
        'sources': [f['tool'] for f in findings],
        'confidence': 1.0  # High confidence (exact CVE match)
    }
    return merged
```

Example:

```
# Input
findings = [
    {'cve_id': 'CVE-2021-44228', 'tool': 'Snyk', 'severity': 10.0},
    {'cve_id': 'CVE-2021-44228', 'tool': 'SonarQube', 'severity': 10.0},
    {'cve_id': 'CVE-2021-44228', 'tool': 'Wiz', 'severity': 10.0}
]

# Output
deduplicated = [
    {
        'cve_id': 'CVE-2021-44228',
        'component': 'log4j-core',
        'version': '2.14.1',
        'severity': 10.0,
        'sources': ['Snyk', 'SonarQube', 'Wiz'],
        'confidence': 1.0
    }
]
```

Pros: - High accuracy (99%+) - Simple implementation - Fast execution

Cons: - Requires CVE IDs (not all findings have CVEs) - Misses findings with different CVE IDs for same vulnerability

1.6.2 3.2 Algorithm 2: Component-Based Deduplication

Input: Multiple findings for same component + version

Algorithm:

```
def deduplicate_by_component(findings):
    """Deduplicate findings by component + version"""
```

```
    component_groups = {}

    for finding in findings:
        component = finding.get('component')
        version = finding.get('version')
        key = f"{component}:{version}"

        if key not in component_groups:
            component_groups[key] = []

        component_groups[key].append(finding)

    deduplicated = []
    for key, group in component_groups.items():
        # Check if findings are similar
        if are_similar(group):
            merged = merge_findings(group)
            deduplicated.append(merged)
        else:
            # Keep separate if not similar
            deduplicated.extend(group)

    return deduplicated

def are_similar(findings):
    """Check if findings are similar enough to merge"""
    # Compare vulnerability descriptions using cosine similarity
    descriptions = [f.get('description', '') for f in findings]
    similarities = []

    for i in range(len(descriptions)):
        for j in range(i+1, len(descriptions)):
            sim = cosine_similarity(descriptions[i], descriptions[j])
            similarities.append(sim)

    # Merge if average similarity > 0.8
    return sum(similarities) / len(similarities) > 0.8 if similarities else False
```

Example:

```
# Input
findings = [
    {
        'component': 'log4j-core',
        'version': '2.14.1',
        'tool': 'Snyk',
        'description': 'Remote code execution vulnerability in log4j'
    },
```

```
    {
        'component': 'log4j-core',
        'version': '2.14.1',
        'tool': 'Wiz',
        'description': 'RCE vulnerability in Apache Log4j'
    }
]

# Similarity calculation
similarity = cosine_similarity(
    'Remote code execution vulnerability in log4j',
    'RCE vulnerability in Apache Log4j'
)
# similarity = 0.87 (> 0.8 threshold)

# Output
deduplicated = [
    {
        'component': 'log4j-core',
        'version': '2.14.1',
        'sources': ['Snyk', 'Wiz'],
        'confidence': 0.87
    }
]
```

Pros: - Works without CVE IDs - Catches similar vulnerabilities with different CVE IDs

Cons: - Lower accuracy (80-90%) - Requires NLP for similarity calculation - Slower execution

1.6.3 3.3 Algorithm 3: Location-Based Deduplication

Input: Multiple SAST findings at same location

Algorithm:

```
def deduplicate_by_location(findings):
    """Deduplicate SAST findings by file + line number"""
    location_groups = {}

    for finding in findings:
        if finding.get('type') != 'SAST':
            continue

        file_path = finding.get('file')
        line_number = finding.get('line')
        key = f"{file_path}:{line_number}"

        if key not in location_groups:
```

```
            location_groups[key] = []

        location_groups[key].append(finding)

    deduplicated = []
    for key, group in location_groups.items():
        # Merge findings at same location
        merged = merge_findings(group)
        deduplicated.append(merged)

    return deduplicated
```

Example:

```
# Input
findings = [
    {
        'type': 'SAST',
        'tool': 'SonarQube',
        'file': 'src/db/query.java',
        'line': 42,
        'vulnerability': 'SQL Injection'
    },
    {
        'type': 'SAST',
        'tool': 'Semgrep',
        'file': 'src/db/query.java',
        'line': 42,
        'vulnerability': 'SQL Injection'
    }
]

# Output
deduplicated = [
    {
        'type': 'SAST',
        'file': 'src/db/query.java',
        'line': 42,
        'vulnerability': 'SQL Injection',
        'sources': ['SonarQube', 'Semgrep'],
        'confidence': 0.95
    }
]
```

Pros: - High accuracy for SAST findings - Simple implementation

Cons: - Only works for SAST findings - Doesn’t work for SCA/CNAPP findings

---

1.7 4. Context Enrichment Techniques

1.7.1 4.1 SBOM Context

Enrich findings with SBOM data:

SAST Finding (without context):

```
{
  "type": "SAST",
  "vulnerability": "Vulnerable Dependency",
  "component": "log4j-core:2.14.1",
  "severity": "CRITICAL"
}
```

SBOM Data:

```
{
  "component": "log4j-core",
  "version": "2.14.1",
  "purl": "pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1",
  "licenses": ["Apache-2.0"],
  "dependency_type": "transitive",
  "parent": "spring-boot-starter-web:2.5.0"
}
```

Enriched Finding:

```
{
  "type": "SAST",
  "vulnerability": "Vulnerable Dependency",
  "component": "log4j-core",
  "version": "2.14.1",
  "purl": "pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1",
  "licenses": ["Apache-2.0"],
  "dependency_type": "transitive",
  "parent": "spring-boot-starter-web:2.5.0",
  "remediation": "Upgrade spring-boot-starter-web to 2.7.0",
  "severity": "CRITICAL"
}
```

Value: Provides remediation guidance (upgrade parent dependency)

1.7.2 4.2 Business Context

Enrich findings with business context:

SCA Finding (without context):

```
{
  "type": "SCA",
  "cve_id": "CVE-2024-1234",
```

```
  "component": "example-lib",
  "version": "1.2.3",
  "cvss": 7.5,
  "severity": "HIGH"
}
```

Business Context:

```
{
  "component": "api-gateway",
  "internet_facing": true,
  "data_sensitive": true,
  "critical_service": true,
  "business_unit": "retail-banking"
}
```

Enriched Finding:

```
{
  "type": "SCA",
  "cve_id": "CVE-2024-1234",
  "component": "example-lib",
  "version": "1.2.3",
  "cvss": 7.5,
  "severity": "HIGH",
  "internet_facing": true,
  "data_sensitive": true,
  "critical_service": true,
  "business_unit": "retail-banking",
  "fixops_risk_score": 78,
  "priority": "HIGH"
}
```

Value: Increases risk score due to exposure and criticality

1.7.3 4.3 Threat Intelligence

Enrich findings with threat intelligence:

CVE Finding (without threat intel):

```
{
  "cve_id": "CVE-2024-5678",
  "component": "spring-core",
  "version": "5.3.10",
  "cvss": 8.1,
  "severity": "HIGH"
}
```

Threat Intelligence:

```
{
  "cve_id": "CVE-2024-5678",
  "epss_score": 0.82456,
  "epss_percentile": 0.97234,
  "kev_flag": true,
  "kev_due_date": "2025-02-15",
  "exploit_available": true,
  "exploit_maturity": "functional"
}
```


Enriched Finding:


```
{
  "cve_id": "CVE-2024-5678",
  "component": "spring-core",
  "version": "5.3.10",
  "cvss": 8.1,
  "severity": "HIGH",
  "epss_score": 0.82456,
  "epss_percentile": 0.97234,
  "kev_flag": true,
  "kev_due_date": "2025-02-15",
  "exploit_available": true,
  "exploit_maturity": "functional",
  "fixops_risk_score": 88,
  "priority": "CRITICAL"
}
```


Value: Increases risk score due to active exploitation

1.8 5. Unified Risk Scoring

1.8.1 5.1 Multi-Scanner Risk Aggregation

AlDeci aggregates risk scores from multiple scanners:

Scanner 1 (Snyk):


```
{
  "cve_id": "CVE-2021-44228",
  "severity": "CRITICAL",
  "cvss": 10.0,
  "snyk_priority_score": 950
}
```


Scanner 2 (Wiz):


```
{
  "cve_id": "CVE-2021-44228",
```

```
  "severity": "CRITICAL",
  "cvss": 10.0,
  "wiz_risk_score": 95
}
```

AlDeci Unified Risk:

```
{
  "cve_id": "CVE-2021-44228",
  "cvss": 10.0,
  "epss_percentile": 0.99876,
  "kev_flag": true,
  "version_lag_days": 1326,
  "internet_facing": true,
  "fixops_risk_score": 92,
  "priority": "CRITICAL",
  "sources": ["Snyk", "Wiz"],
  "confidence": 1.0
}
```

Value: Single risk score across all scanners

1.8.2 5.2 Risk Score Normalization

AlDeci normalizes scanner-specific scores to FixOpsRisk (0-100):

Scanner-Specific Scores: - Snyk Priority Score: 0-1000 - Wiz Risk Score: 0-100 - SonarQube Severity: Info, Minor, Major, Critical, Blocker - Semgrep Severity: Low, Medium, High, Critical

Normalization Formula:

```
def normalize_score(scanner, score):
    """Normalize scanner-specific score to 0-100"""
    if scanner == 'Snyk':
        return (score / 1000) * 100
    elif scanner == 'Wiz':
        return score  # Already 0-100
    elif scanner == 'SonarQube':
        severity_map = {'Info': 10, 'Minor': 30, 'Major': 60, 'Critical': 85, 'Blocker': 95}
        return severity_map.get(score, 50)
    elif scanner == 'Semgrep':
        severity_map = {'Low': 20, 'Medium': 50, 'High': 75, 'Critical': 90}
        return severity_map.get(score, 50)
    else:
        return 50  # Default
```

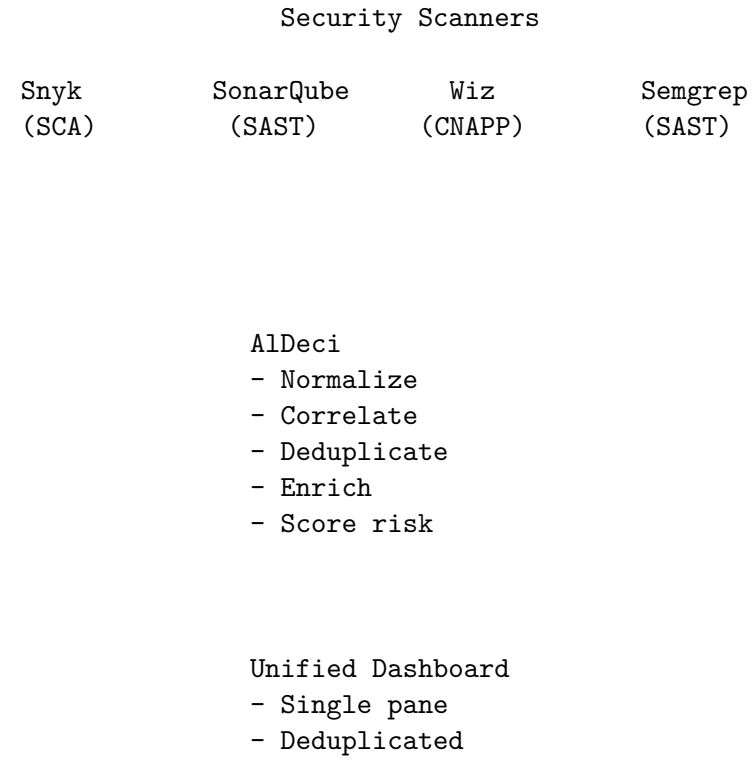
Example:

```python
# Snyk Priority Score: 950
normalized_snyk = normalize_score('Snyk', 950)
# normalized_snyk = 95

# SonarQube Severity: Critical
normalized_sonarqube = normalize_score('SonarQube', 'Critical')
# normalized_sonarqube = 85

# Unified FixOpsRisk score
fixops_risk_score = calculate_fixops_risk(
    cvss=10.0,
    epss_percentile=0.99876,
    kev_flag=True,
    version_lag_days=1326,
    internet_facing=True
)
# fixops_risk_score = 92
```

1.9 6. Implementation Patterns

1.9.1 6.1 Pattern 1: Centralized Correlation

Architecture:



```
                - Prioritized
```

Implementation:

```
# Push findings from each scanner to AlDeci
curl -X POST https://api.devops.ai/products/aldeci/inputs/sarif \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@snyk-scan.sarif" \
  -F "source=snyk"

curl -X POST https://api.devops.ai/products/aldeci/inputs/sarif \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@sonarqube-scan.sarif" \
  -F "source=sonarqube"

curl -X POST https://api.devops.ai/products/aldeci/inputs/sarif \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@wiz-scan.sarif" \
  -F "source=wiz"

# AlDeci automatically correlates and deduplicates
# Query unified findings
curl -X GET https://api.devops.ai/products/aldeci/findings/unified \
  -H "Authorization: Bearer $TOKEN"
```

Pros: - Single source of truth - Automatic correlation - No scanner-specific logic in CI/CD

Cons: - Requires AlDeci deployment - Additional infrastructure

1.9.2 6.2 Pattern 2: Pipeline-Based Correlation

Architecture:

```
                    CI/CD Pipeline

    Snyk    → SonarQube →    Wiz    →  Correlate



                                          Unified Report
```

Implementation:

```
# .github/workflows/security.yml
jobs:
```

```yaml
  security:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Run Snyk
        run: snyk test --sarif > snyk.sarif

      - name: Run SonarQube
        run: sonar-scanner --sarif > sonarqube.sarif

      - name: Run Wiz
        run: wiz scan --sarif > wiz.sarif

      - name: Correlate findings
        run: |
          aldeci correlate \
            --input snyk.sarif \
            --input sonarqube.sarif \
            --input wiz.sarif \
            --output unified-findings.json

      - name: Upload unified findings
        uses: actions/upload-artifact@v3
        with:
          name: unified-findings
          path: unified-findings.json
```

Pros: - No additional infrastructure - Correlation happens in pipeline

Cons: - Requires AlDeci CLI in pipeline - Slower pipeline execution

---

1.10 7. Tool-Specific Integration

1.10.1 7.1 Snyk Integration

SARIF Export:

```
snyk test --sarif-file-output=snyk.sarif
```

Push to AlDeci:

```
curl -X POST https://api.devops.ai/products/aldeci/inputs/sarif \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@snyk.sarif" \
  -F "source=snyk" \
  -F "scan_type=SCA"
```

1.10.2 7.2 SonarQube Integration

SARIF Export:

```
sonar-scanner \
  -Dsonar.projectKey=my-project \
  -Dsonar.sources=. \
  -Dsonar.host.url=https://sonarqube.example.com \
  -Dsonar.login=$SONAR_TOKEN \
  -Dsonar.sarif.output=sonarqube.sarif
```

Push to AIDeci:

```
curl -X POST https://api.devops.ai/products/aldeci/inputs/sarif \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@sonarqube.sarif" \
  -F "source=sonarqube" \
  -F "scan_type=SAST"
```

1.10.3 7.3 Wiz Integration

SARIF Export:

```
wiz scan --format sarif > wiz.sarif
```

Push to AIDeci:

```
curl -X POST https://api.devops.ai/products/aldeci/inputs/sarif \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@wiz.sarif" \
  -F "source=wiz" \
  -F "scan_type=CNAPP"
```

1.10.4 7.4 Semgrep Integration

SARIF Export:

```
semgrep --config=auto --sarif > semgrep.sarif
```

Push to AIDeci:

```
curl -X POST https://api.devops.ai/products/aldeci/inputs/sarif \
  -H "Authorization: Bearer $TOKEN" \
  -F "file=@semgrep.sarif" \
  -F "source=semgrep" \
  -F "scan_type=SAST"
```

1.11 8. Best Practices

1.11.1 8.1 Scanner Selection

1. **Use complementary scanners** - SAST + SCA + CNAPP for comprehensive coverage
2. **Avoid redundant scanners** - Don't use 3 SAST tools (diminishing returns)
3. **Prioritize accuracy** - Choose scanners with low false-positive rates
4. **Consider cost** - Balance coverage vs. licensing costs
5. **Evaluate integration** - Choose scanners with SARIF export

1.11.2 8.2 Correlation Configuration

1. **Set confidence thresholds** - Auto-deduplicate high confidence (≥ 0.9)
2. **Review medium confidence** - Human review for medium confidence (0.7-0.9)
3. **Keep low confidence separate** - Don't auto-deduplicate low confidence (< 0.7)
4. **Tune similarity thresholds** - Adjust based on false positive/negative rates
5. **Monitor correlation metrics** - Track deduplication rate, false positives

1.11.3 8.3 Risk Prioritization

1. **Use unified risk scores** - Don't rely on scanner-specific scores
2. **Enrich with context** - Add business context, threat intel
3. **Prioritize by FixOpsRisk** - Focus on high-risk findings first
4. **Track MTTR by risk level** - Measure remediation velocity
5. **Generate regular reports** - Weekly unified risk reports

1.11.4 8.4 Remediation Workflow

1. **Create single ticket** - One ticket per deduplicated finding
2. **Link to all sources** - Reference all scanners that detected the finding
3. **Include unified risk score** - FixOpsRisk score in ticket
4. **Attach evidence bundle** - Signed evidence for audit
5. **Track remediation** - Monitor MTTR and SLA compliance

1.12 9. Case Studies

1.12.1 9.1 Case Study: Financial Services Organization

Profile: - 500+ microservices - 3 security scanners (Snyk, SonarQube, Wiz) - 12,000 open findings - 40% duplicate findings

Challenge: - Security team overwhelmed with duplicates - Unclear which findings to prioritize - Manual deduplication taking 2 days per week

AlDeci Implementation: - Integrated all 3 scanners with AlDeci - Configured CVE-based and component-based deduplication - Enriched findings with EPSS, KEV, business context - Generated unified risk scores

Results: - Reduced findings from 12,000 to 7,200 (40% deduplication) - Identified 180 critical findings (FixOpsRisk ≥ 85) - Reduced manual deduplication time from 2 days to 2 hours per week - Reduced MTTR for critical findings from 45 days to 7 days

1.12.2 9.2 Case Study: SaaS Platform Provider

Profile: - 200+ services - 4 security scanners (Snyk, Semgrep, Trivy, Wiz) - 8,000 open findings - 50% duplicate findings

Challenge: - Multiple scanners generating overlapping findings - No unified view of security posture - Difficult to track remediation progress

AlDeci Implementation: - Integrated all 4 scanners with AlDeci - Configured multi-dimensional correlation (CVE + component + location) - Enriched findings with SBOM data and business context - Generated unified dashboard

Results: - Reduced findings from 8,000 to 4,000 (50% deduplication) - Unified view of security posture across all scanners - Reduced false-positive remediation by 62% - Improved remediation velocity by 3x

---

DevOps.ai | Sydney, Australia | https://devops.ai | contact@devops.ai